

TAREA AUTÓNOMA

Sistema de Reservas de Hotel con Microservicios

Asignatura:	Microservicios
Docente:	Ing. Viviana Flores, MSc.
Carrera:	Ingeniería en Sistemas / Informática
Modalidad:	Individual o en parejas
Ponderación:	10 puntos

1. Objetivo

Construir un sistema de reservas hoteleras basado en arquitectura de microservicios, aplicando los patrones de comunicación entre servicios estudiados en clase: diseño de contratos REST, RestTemplate, WebClient y Resilience4j (Circuit Breaker, Retry y Fallback).

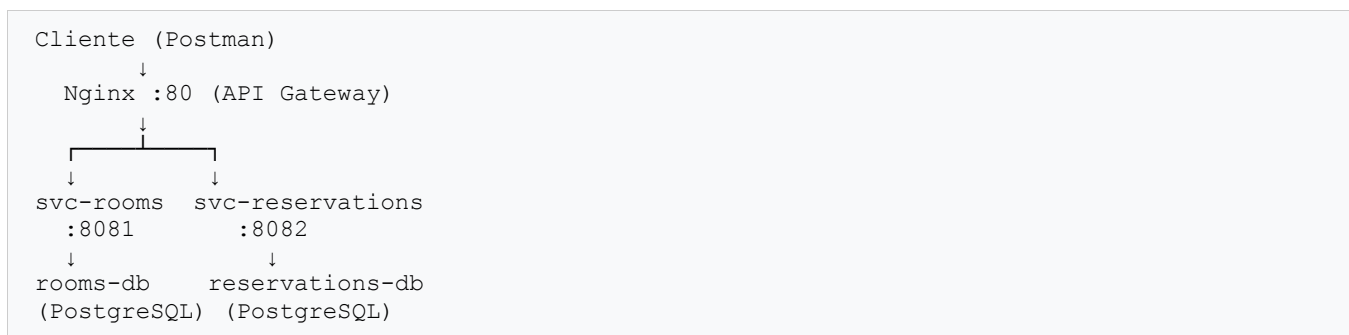
2. Contexto del Negocio

El Hotel "Los Andes" necesita digitalizar su sistema de reservas. Se requiere desarrollar dos microservicios independientes que se comuniquen entre sí:

- svc-rooms — Gestiona el catálogo de habitaciones del hotel y su disponibilidad.
- svc-reservations — Gestiona las reservas de los clientes. Consume svc-rooms para verificar disponibilidad y obtener datos de la habitación al momento de crear una reserva.

3. Arquitectura Requerida

El sistema debe implementarse con la siguiente arquitectura de microservicios:



Reglas de arquitectura obligatorias:

- Cada microservicio debe tener su propia base de datos independiente (Database per Service).

- La comunicación entre svc-reservations y svc-rooms es exclusivamente por HTTP REST.
- Todo el sistema debe levantarse con un solo comando: docker-compose up -d --build.
- Nginx actúa como API Gateway enrutando peticiones al servicio correcto según la URI.

4. Microservicio 1 — svc-rooms

Este microservicio gestiona el inventario de habitaciones del hotel. Es el proveedor de información para svc-reservations.

4.1 Estructura de paquetes

```
com.uti.svcrooms
├── config/
├── controller/
├── dto/
├── exception/
├── mapper/
├── model/
├── repository/
├── service/
└── impl/
```

4.2 Modelo de datos — Entidad Room

Campo	Tipo	Restricción	Descripción
id	Long	PK, autoincremental	Identificador único
roomNumber	String	NOT NULL, UNIQUE	Número de habitación (ej: 101)
type	Enum	NOT NULL	SINGLE, DOUBLE, SUITE
pricePerNight	Double	NOT NULL	Precio por noche en USD
totalCapacity	Integer	NOT NULL	Total de habitaciones de este tipo
availableRooms	Integer	NOT NULL	Habitaciones disponibles actualmente
floor	Integer	—	Piso de la habitación
description	String	—	Descripción adicional

4.3 Endpoints requeridos

Método	URI	Descripción	Código éxito
GET	/api/v1/rooms	Listar todas las habitaciones	200 OK
GET	/api/v1/rooms/{id}	Obtener habitación por ID	200 OK
POST	/api/v1/rooms	Registrar nueva habitación	201 Created
GET	/api/v1/rooms/{id}/availability	Consultar disponibilidad	200 OK
PATCH	/api/v1/rooms/{id}/availability	Actualizar disponibilidad	200 OK

4.4 Reglas de negocio

- roomNumber debe ser único en todo el sistema — lanzar 409 Conflict si ya existe.
- availableRooms no puede ser negativo — lanzar 422 Unprocessable Entity.
- availableRooms no puede superar totalCapacity — lanzar 422 Unprocessable Entity.
- Todas las respuestas de error deben tener formato uniforme: timestamp, status, error, message, path.

4.5 Formato de respuesta de disponibilidad

```
{
  "roomId": 1,
  "available": true,
  "availableRooms": 3
}
```

5. Microservicio 2 — svc-reservations

Este microservicio gestiona las reservas de los huéspedes. Es el consumidor — llama a svc-rooms para verificar disponibilidad y obtener datos de la habitación.

5.1 Estructura de paquetes

```
com.uti.svcreservations
├── config/           ← AppConfig con beans RestTemplate y WebClient
├── controller/
├── dto/
├── exception/
├── mapper/
├── model/
├── repository/
├── service/
│   └── impl/
└── client/          ← RoomsRestTemplateClient y RoomsWebClient
```

5.2 Modelo de datos — Entidad Reservation

Campo	Tipo	Restricción	Descripción
id	Long	PK, autoincremental	Identificador único
roomId	Long	NOT NULL	ID de la habitación en svc-rooms
guestName	String	NOT NULL	Nombre completo del huésped
guestEmail	String	NOT NULL	Email del huésped
checkInDate	LocalDate	NOT NULL	Fecha de entrada
checkOutDate	LocalDate	NOT NULL	Fecha de salida
status	Enum	NOT NULL	ACTIVE, COMPLETED, CANCELLED
totalNights	Integer	Calculado	checkOutDate - checkInDate
createdAt	LocalDateTime	Automático	Timestamp de creación

5.3 Endpoints requeridos

Método	URI	Descripción	Código éxito
POST	/api/v1/reservations	Crear reserva	201 Created
GET	/api/v1/reservations/{id}	Obtener reserva con datos del cuarto	200 OK
GET	/api/v1/reservations/guest/{email}	Reservas por email del huésped	200 OK
PATCH	/api/v1/reservations/{id}/checkout	Registrar salida del huésped	200 OK

5.4 Reglas de negocio

- Al crear una reserva: verificar disponibilidad en svc-rooms. Si available=false lanzar 422 Unprocessable Entity.
- Un huésped no puede tener dos reservas ACTIVE para la misma habitación — lanzar 422.
- checkOutDate debe ser posterior a checkInDate — lanzar 400 Bad Request si no se cumple.
- totalNights se calcula automáticamente: checkOutDate - checkInDate en días.
- Solo se puede hacer checkout de una reserva con status ACTIVE.

5.5 Formato de respuesta enriquecida (POST /reservations)

```
{
  "id": 1,
  "roomId": 1,
  "guestName": "Maria Torres",
  "guestEmail": "maria.torres@email.com",
  "checkInDate": "2026-07-01",
  "checkOutDate": "2026-07-05",
  "totalNights": 4,
  "status": "ACTIVE",
  "createdAt": "2026-06-24T10:30:00",
  "roomNumber": "101",           <- datos de svc-rooms via WebClient
  "roomType": "DOUBLE",         <- datos de svc-rooms via WebClient
  "pricePerNight": 85.00        <- datos de svc-rooms via WebClient
}
```

6. Requisitos Técnicos Obligatorios

6.1 Comunicación entre servicios

svc-reservations debe consumir svc-rooms usando ambos clientes HTTP según se indica:

Método en svc-reservations	Cliente HTTP	Endpoint svc-rooms	Propósito
createReservation()	RestTemplate	GET /rooms/{id}/availability	Verificar disponibilidad
createReservation()	WebClient	GET /rooms/{id}	Datos del cuarto para respuesta
getReservationById()	RestTemplate	GET /rooms/{id}	Enriquecer respuesta

getReservationsByEmail()	WebClient	GET /rooms/{id}	Enriquecer respuesta
checkout()	RestTemplate	GET /rooms/{id}	Datos del cuarto para respuesta

6.2 Resiliencia con Resilience4j

Implementar en svc-reservations los siguientes patrones de resiliencia para proteger las llamadas a svc-rooms:

Patrón	Configuración mínima	Comportamiento esperado
Circuit Breaker	sliding-window-size=5 failure-rate-threshold=50% wait-duration=10s	Después de 3 fallos seguidos, el circuito se abre y las llamadas van directo al fallback sin esperar timeout
Retry	max-attempts=3 wait-duration=2s	Si svc-rooms falla, reintenta hasta 3 veces con 2 segundos de espera entre intentos
Fallback	Método alternativo requerido	Cuando Circuit Breaker está abierto: devuelve la reserva con mensaje 'Room information temporarily unavailable'

6.3 Docker Compose requerido

El archivo docker-compose.yml debe levantar los siguientes servicios:

- rooms-db — PostgreSQL para svc-rooms (puerto externo: 5433)
- reservations-db — PostgreSQL para svc-reservations (puerto externo: 5434)
- svc-rooms — Microservicio de habitaciones (puerto: 8081)
- svc-reservations — Microservicio de reservas (puerto: 8082)
- nginx-gateway — API Gateway Nginx (puerto: 80)

7. Entregables

#	Entregable	Descripción
1	Código fuente	Repositorio GitHub con ambos microservicios (svc-rooms y svc-reservations) con historial de commits
2	docker-compose.yml	Funcional — debe levantar todo el sistema con un solo comando: docker-compose up -d --build
3	Capturas Postman	Capturas con todas las pruebas organizadas por carpetas (svc-rooms y svc-reservations)
4	Video demostrativo	Máximo 5 minutos mostrando: sistema funcionando, prueba de Resilience4j (parar svc-rooms y demostrar fallback y recuperación)

8. Pruebas Mínimas a Demostrar

Las siguientes pruebas deben estar documentadas en la colección Postman y demostradas en el video:

#	Prueba	Resultado esperado
---	--------	--------------------

1	Registrar 3 habitaciones (SINGLE, DOUBLE, SUITE)	201 Created por cada una
2	Crear reserva válida para habitación disponible	201 Created — respuesta enriquecida con datos del cuarto (roomNumber, roomType, pricePerNight)
3	Crear reserva para habitación sin disponibilidad	422 Unprocessable Entity — mensaje: 'Room is not available'
4	Crear reserva duplicada (mismo huésped, mismo cuarto, ACTIVE)	422 Unprocessable Entity — mensaje de préstamo duplicado
5	Consultar reserva por ID con svc-rooms activo	200 OK — datos completos del cuarto incluidos
6	Registrar checkout de reserva ACTIVE	200 OK — status cambia a COMPLETED
7	Intentar checkout de reserva ya COMPLETED	422 Unprocessable Entity
8	Parar svc-rooms y crear reserva	503 Service Unavailable — mensaje del fallback de Resilience4j
9	Consultar reserva con svc-rooms caído	200 OK — roomNumber: 'Room information temporarily unavailable'
10	Reiniciar svc-rooms y verificar recuperación automática	El sistema funciona normalmente — Circuit Breaker cierra

9. Buenas Prácticas Obligatorias

9.1 Estructura del código

- Nombres de clases, métodos y variables en inglés.
- Comentarios explicativos en cada clase y método.
- DTOs completamente separados de las entidades JPA — nunca exponer la entidad directamente.
- Patrón Interface + Implementación en la capa Service.
- GlobalExceptionHandler centralizado — ningún Controller tiene bloques try/catch.

9.2 Calidad del código

- Validaciones con Bean Validation (@NotNull, @NotBlank, @Email, @Min) en los DTOs de entrada.
- Logging con @Slf4j en Service y Client — mensajes INFO para flujo normal, WARN para fallbacks, ERROR para errores inesperados.
- Formato uniforme de error en todas las respuestas de error: { timestamp, status, error, message, path }.

10. Forma de Entrega

Forma de entrega:

- Subir el enlace del repositorio GitHub (debe ser público o compartido con la docente).
- Adjuntar documento con las capturas solicitadas (pruebas en Postman y contenedores en Docker).
- Subir el enlace del video demostrativo (YouTube, Google Drive u otra plataforma).

Nota: Los trabajos entregados fuera del plazo establecido o que no cumplan con los entregables mínimos no serán evaluados.

RÚBRICA DE EVALUACIÓN — TAREA AUTÓNOMA

Criterio	Excelente (10 - 9)	Bueno (8.9 - 7.5)	Suficiente (7.4 - 6)	Insuficiente (< 6)	Peso
1. Arquitectura y Docker	Ambos servicios levantan con docker-compose up, Nginx funciona como gateway, BDs completamente separadas	Levantar con errores menores, Nginx configurado pero con problemas de enrutamiento	Levantar manualmente desde IntelliJ sin Docker, sin Nginx	No levanta ningún servicio o no compila	20%
2. svc-rooms	Todos los endpoints funcionan correctamente, validaciones completas, manejo de errores con formato uniforme	Endpoints funcionan, faltan algunas validaciones (ej: roomNumber duplicado no controlado)	Solo endpoints básicos GET y POST sin validaciones	No funciona, no compila o no tiene manejo de errores	20%
3. svc-reservations — CRUD	Todos los endpoints funcionan, reglas de negocio implementadas, respuesta enriquecida con datos del cuarto	Endpoints funcionan pero faltan reglas de negocio (ej: no valida disponibilidad)	Solo POST y GET básico sin enriquecimiento de respuesta	No funciona o no implementa la comunicación con svc-rooms	20%
4. RestTemplate y WebClient	Ambos implementados correctamente en los métodos indicados, comentarios explicando la diferencia entre ambos	Ambos implementados pero usados indistintamente (no en los métodos indicados)	Solo uno implementado (RestTemplate O WebClient, no ambos)	No implementados o copiados sin entender su función	20%
5. Resilience4j	Circuit Breaker, Retry y Fallback funcionan correctamente. Demostrado en video con svc-rooms caído. Logs visibles	Circuit Breaker y Fallback funcionan pero Retry no está configurado correctamente	Solo Fallback implementado, sin Circuit Breaker ni Retry	No implementado o no funciona al detener svc-rooms	20%

Nota Final	Sobre 10	Calificación
Suma ponderada de todos los criterios (cada criterio tiene peso del 20%)	/ 10	